EUI OMR Engine

Industrial-Grade Optical Mark Recognition Platform

EUI Advanced Agentic Coding Team

April 16, 2026

Contents

1 Abstract **3**

2 System Architecture **3**
2.1 Architectural Components 3
2.2 Concurrency Model 3

3 Computer Vision & OMR Pipeline **3**
3.1 Corner Detection & Orthorectification 4
3.2 Extraction Algorithms 4

4 UI/UX Design & User Flow **4**
4.1 Dashboard Interface 4
4.2 Systemic Calibration 4

5 Data Management & Flow **5**
5.1 Project Shadowing (Data Guard) 5
5.2 Memory Management (Disk Caching) 5

6 Usage Guide **5**
6.1 Initial Setup 5
6.2 Processing and Review 5

7 Installation & Requirements **5**
7.1 Dependencies 5
7.2 Production Build 6

8 Known Limitations & Future Work **6**

1 Abstract

The EUI OMR Engine is a high-performance, independent Optical Mark Recognition (OMR) platform designed for large-scale academic assessments. Unlike traditional OMR systems that require proprietary hardware and specialized paper, this system utilizes Computer Vision (OpenCV) and modern desktop processing to extract data from standard A4 paper scanned by commercial office equipment.

The platform features a SaaS-inspired dashboard, an asynchronous multi-threaded processing engine, and a robust "Data Guard" project management system. This manual provides a holistic technical overview of the system architecture, underlying algorithms, and operational workflows.

2 System Architecture

The EUI OMR Engine follows a modified **Model-View-Controller (MVC)** pattern, optimized for low-latency asynchronous processing.

2.1 Architectural Components

- **Model Layer**: Represented by `ProjectManager` and `GradingResult` schemas. It handles the serialization of application state to `project_state.json` and manages the physical mirroring of assets (PDFs, Excel rosters).
- **View Layer**: A PySide6-based interface consisting of the `MainWindow`, a unified `Sidebar`, and a `SlidingStackedWidget`. High-performance UI transitions are handled via an integrated "Sliding Fade" animation engine.
- **Engine Layer (Controller)**: The OMR pipeline (`OMRGrader` and `calibrate.py`) which executes the Computer Vision logic.
- **Worker Layer**: Asynchronous `QThread` implementations (`PDFGraderWorker`, `AutoTuneWorker`) that offload heavy computations, preventing GUI freezing.

2.2 Concurrency Model

To maintain 60FPS UI responsiveness, the engine utilizes a signal-slot mechanism for worker communication.

1. **Producer**: The UI initiates a thread with target scanning parameters.
2. **Consumer**: The worker loads the PDF via `PyMuPDF`, converts pages to pixel buffers, and passes them to the Grader.
3. **Feedback**: Results and progress are emitted back to the main thread for real-time dashboard updates.

3 Computer Vision & OMR Pipeline

The engine's reliability stems from a rigorous image-processing pipeline that transforms raw scanner output into clean, predictable data structures.

3.1 Corner Detection & Orthorectification

The system uses `detect_corners()` in `calibrate.py` to align sheets.

- **Marker Heuristic**: Detects four physical markers. One "Anchor" (Rectangle, aspect ratio ≈ 2.0) and three "Squares" (aspect ≈ 1.0).
- **Orientation Logic**: By identifying the geometric center of the four markers and the specific location of the Anchor, the system detects and automatically corrects 180-degree rotations.
- **Perspective Warp**: Uses `cv2.getPerspectiveTransform` to map the sheet into a fixed A4 resolution (1654x2339 pixels) at 200 DPI.

3.2 Extraction Algorithms

Once aligned, the `OMRGrader` processes specific Regions of Interest (ROIs).

1. **Thresholding**: Applies `ADAPTIVE_THRESH_GAUSSIAN_C` to handle varying lighting and scanner noise.
2. **Grid Slicing**: Divides ROIs into sub-cells while applying a 3-pixel internal padding safety margin to ignore bubble borders.
3. **ID Peak Detection**: Instead of simple binary thresholding, the ID logic uses a Peak Detection algorithm. It compares signal strength across columns to distinguish actual pencil marks from messy erasures or faint artifacts.

4 UI/UX Design & User Flow

4.1 Dashboard Interface

The UI is built on a "SaaS-style" 8px grid system with a premium dark-mode aesthetic (`#121212`).

- **Sidebar**: A collapsible navigation hub providing one-click access to Home, Setup, and Dashboard views.
- **View Transitions**: Implements a `SlidingStackedWidget` that performs synchronized position and opacity animations, ensuring a high-end feel.
- **Card-Based Layout**: Information is grouped into interactive containers, reducing cognitive load for the operator.

4.2 Systemic Calibration

The "Phase 1: Auto-Calibration" step (`AutoTuneWorker`) automatically selects random sample pages from the target PDF to find the optimal mark sensitivity. This eliminates manual configuration for different paper weights or pencil darkness.

5 Data Management & Flow

5.1 Project Shadowing (Data Guard)

To ensure project portability, the `ProjectManager` implements a "Shadowing" policy:

1. User selects a PDF or Excel file from any location on their computer.
2. The system *mirrors* these files into the project's `/imports` directory.
3. All internal references use **Relative Paths**.

Design Decision: This allows users to move the entire project folder across devices (via USB or Cloud) without breaking file links or losing session data.

5.2 Memory Management (Disk Caching)

During large batch processing (e.g., 500+ sheets), the engine avoids RAM exhaustion by:

- Slicing the visual crops from the sheet images.
- Writing these crops to the `/crops` directory as compressed JPEGs.
- Clearing the pixel buffers from memory immediately after save.

6 Usage Guide

6.1 Initial Setup

1. **Import Roster**: Select an Excel file. The system will detect columns; assign the "ID Column" and a target "Grade Output" column.
2. **Define Keys**: Managed via the "Answer Key Manager". Each version (A-F) can be configured manually or "scanned" from a model sheet.
3. **Target PDF**: Load the scan batch.

6.2 Processing and Review

The "Dashboard" shows real-time progress. Any sheet with low-confidence marks, double marks, or invalid IDs is flagged with a **Needs Review** status. The operator can double-click any row to open the `ReviewModal`, which displays the actual physical crops side-by-side with the extracted values for verification.

7 Installation & Requirements

7.1 Dependencies

The system requires Python 3.9+ and the following libraries:

```
pip install PySide6 opencv-python numpy PyMuPDF pandas openpyxl
```

7.2 Production Build

To generate a standalone Windows executable (`.exe`):

```
pyinstaller EUI_OMR_Engine.spec --clean
```

8 Known Limitations & Future Work

- **Handwritten OCR**: Currently, the system identifies the ID marks but does not perform OCR on the handwritten signature area; this area is provided only for visual verification.
- **Sheet Skew**: While 180-degree rotation and perspective warping are supported, extreme folds in the paper may cause local nonlinear distortions.
- **Feature Roadmap**: Implementation of a cloud-sync module for multi-station grading environments.